

1 Problem Editorial: ESM

1.1 Problem Statement Overview

The problem appears to involve finding patterns in an array of integers, with three different queries (cases) that need to be answered based on the value of c :

- **Case 1:** Count geometric progression triples where consecutive elements satisfy $v[i] \times v[i + 1] = v[i + 2]$
- **Case 2:** Given the last element $v[n]$, find the maximum index j where two divisors of $v[n]$ both appear at positions $\leq j$
- **Case 3:** For each element $v[x]$, find the maximum index $j < x$ such that $v[x]$ can be written as a product of two previously seen elements, then sum all such maximum indices

1.2 Case 1: Counting Geometric Triples

The solution for the first case is straightforward. We iterate through all valid starting positions i where $1 \leq i \leq n - 2$ and check if:

$$v[i] \times v[i + 1] = v[i + 2]$$

```
if(c == 1)
{
    int ans = 0;
    for(int i = 1; i + 2 <= n; i++)
        if(1LL * v[i] * v[i+1] == 1LL * v[i+2])
            ans++;
    cout << ans << '\n';
}
```

Key observation: Since $v[i] \times v[i + 1]$ can be large, we use 64-bit integer multiplication (1LL*) to prevent overflow. This case runs in $O(n)$ time.

1.3 Cases 2 and 3: Divisor-Based Approach

Both cases 2 and 3 use a clever technique of tracking the last two occurrences of each value using two auxiliary arrays.

1.3.1 Tracking Previous Occurrences

The code maintains two arrays:

- $fr[x]$: stores the most recent (last) position where value x appears
- $fr2[x]$: stores the second most recent position where value x appears

When processing element $v[i]$, these arrays are updated as follows:

```
fr2[v[i]] = fr[v[i]];
fr[v[i]] = i;
```

This allows us to quickly find the two most recent positions where any value occurred.

1.3.2 Finding Valid Divisor Pairs

For a target value V , we need to find all pairs $(d, V/d)$ where both d and V/d have appeared in the array. We iterate over all divisors i of V :

for $i = 1$ to $\sqrt{V}$: if $V \% i == 0$: consider $(i, V/i)$

The maximum index j where both divisors appear is given by:

$\max(\min(\text{fr}[i], \text{fr}[V/i]))$ over all valid divisor pairs

For the special case where $i = V/i$ (perfect square), we need the second most recent occurrence stored in $\text{fr2}[i]$.

```
for(int i = 1; i * i <= v[x]; i++)
    if(v[x] % i == 0)
    {
        if(i != v[x]/i && fr[i] && fr[v[x]/i])
            maxi = max(maxi, min(fr[i], fr[v[x]/i]));
        if(i == v[x]/i && fr2[i])
            maxi = max(maxi, fr2[i]);
    }
```

1.4 Case 2: Single Query Version

In Case 2, we preprocess the entire array once to build fr and fr2 , then answer a single query for $v[n]$:

```
for(int i = 1; i < n; i++)
{
    fr2[v[i]] = fr[v[i]];
    fr[v[i]] = i;
}
int maxi = 0;
for(int i = 1; i * i <= v[n]; i++)
    if(v[n] % i == 0)
    {
        if(i != v[n]/i && fr[i] && fr[v[n]/i])
            maxi = max(maxi, min(fr[i], fr[v[n]/i]));
        if(i == v[n]/i && fr2[i])
            maxi = max(maxi, fr2[i]);
    }
cout << maxi << '\n';
```

The answer is the maximum index $j < n$ where $v[n]$ can be expressed as a product of two previously seen values.

1.5 Case 3: Cumulative Sum Version

Case 3 processes the array sequentially, maintaining the same fr and fr2 arrays. For each position x , we find the maximum valid index and add it to the running total:

```
long long ans = 0;
for(int x = 1; x <= n; x++)
{
    int maxi = 0;
    // ... divisor checking logic ...
    ans += maxi;
    fr2[v[x]] = fr[v[x]];
    fr[v[x]] = x;
}
cout << ans << '\n';
```

Key insight: Notice that we process the divisor check **before** updating fr and fr2 with the current element. This ensures we only consider positions $j < x$, maintaining the required constraint.

1.6 Complexity Analysis

- **Time Complexity:**

- Case 1: $O(n)$
- Case 2: $O(n + \sqrt{v[n]})$
- Case 3: $O(n \cdot \sqrt{V_{\max}})$, where $V_{\max}$ is the maximum value in the array

- **Space Complexity:** $O(M)$ where M is the maximum possible value (100001 based on array bounds), for storing fr and fr2 arrays

The divisor iteration runs in $O(\sqrt{V})$ per query, which is efficient for the given constraints.

1.7 Important Implementation Notes

1. **64-bit arithmetic:** Always use 1LL* when multiplying integers to avoid overflow when checking geometric progressions.
2. **Divisor pair handling:** When $d = V/d$, we're dealing with a perfect square. In this case, we cannot use fr[d] and fr[V/d] (they're the same array), so we must use fr2[d] instead.
3. **Order of operations:** In Case 3, the update to fr/fr2 must happen *after* the query for that position, otherwise the current element would incorrectly be considered as a valid divisor pair.
4. **Initialization:** The fr array is initialized to 0 (indicating "not seen"), and we must check fr[i] $\neq 0$ before using its value.